Conditional PixelCNN++ for Image Classification and Generation

Avya Malhotra
University Of British Columbia
avya@student.ubc.ca

Abstract

I implement a class-conditional extension of PixelCNN++ for both image generation and classification. PixelCNN++ is a state-of-the-art autoregressive generative model that we condition on class labels to enable the generation of category-specific images and to perform classification. This model incorporates class information via input concatenation and appends a classification head to directly predict labels. I trained the model on a custom dataset consisting of 4,160 labeled training images divided into four categories, along with 520 labeled validation images and 520 unlabeled test images. The model is trained for 300 epochs using a joint loss that combines image reconstruction (via a discretized logistic mixture likelihood) and classification. The final model achieves a test classification accuracy of $\sim 79\%$ and a Fréchet Inception Distance (FID) of ~ 24. I analyzed training dynamics using bits-per-dimension (BPD) and accuracy curves and perform an ablation study to explore the effects of architectural and hyperparameter choices.

1 Introduction

Generative image models with tractable likelihood—such as PixelCNN and PixelCNN++—model the joint distribution of image pixels autoregressively [1]. In particular, PixelCNN++ represents

$$p(\mathbf{x}) = \prod_i p(x_i \mid x_{<i}),$$

using convolutional neural networks with masked filters to ensure each pixel x_i is conditioned only on previous pixels (following a raster scan order). Architectural modifications like multi-scale residual layers and discretized logistic mixture outputs yield state-of-the-art log-likelihoods on datasets [1]. In this work, I extend PixelCNN++ to be *class-conditional* [3] so that it can generate images based on a given class label as well as perform image classification. Conditional PixelCNN models were initially proposed for class-conditioned generation [3]; the key idea is to supply the class information y so that the conditional distribution becomes:

$$p(\mathbf{x} \mid y) = \prod_i p(x_i \mid x_{<i}, y).$$

A trained conditional PixelCNN can also be inverted to perform zero-shot classification by selecting the label that maximizes the likelihood $p(\mathbf{x} \mid y)$.

In this project, instead of relying solely on the generative likelihood for classification, I integrate an explicit classifier within the PixelCNN++ architecture, yielding a multi-task model with shared representations.

NeurIPS 2023.

2 Model

2.1 PixelCNN++ Architecture

My starting point is the PixelCNN++ model [1], which uses a deep convolutional network with masked kernels that respect the autoregressive order. The network is built from multiple residual blocks arranged hierarchically, with downsampling and upsampling paths similar to a U-Net to capture multi-scale structure. Shortcut connections and dropout further aid in optimization and regularization. Instead of using a 256-way softmax per pixel, PixelCNN++ outputs a discretized mixture of logistic distributions. Formally, if the model predicts M mixture components for a given pixel intensity v with component weights π_m, means μ_m, and scales s_m, then the conditional likelihood is given by:

$$p(v \mid \text{context}) \;=\; \sum_{m=1}^{M} \pi_m \left[\sigma\left(\frac{v + 0.5 - \mu_m}{s_m} \right) - \sigma\left(\frac{v - 0.5 - \mu_m}{s_m} \right) \right], \tag{1}$$

where $\sigma(\cdot)$ is the logistic sigmoid function. The overall log-likelihood for an image $\mathbf{x}$ with D pixels is

$$\log p(\mathbf{x}) = \sum_{i=1}^{D} \log p(x_i \mid x_{<i}),$$

and the corresponding negative log-likelihood (NLL) serves as our primary loss for generation.

2.2 Class Conditioning Mechanism

To incorporate the class label y, I provide the label as an extra input. Although the original PixelCNN++ [1] introduced class conditioning by adding a learned, class-dependent bias to each convolutional layer, I opted for a simpler method: *input concatenation*. Here, the class y is encoded as a one-hot vector and spatially replicated, then concatenated with the 3-channel input image. This produces an input of dimensions $32 \times 32 \times (3)$, ensuring that class context is available from the first layer.

2.3 Classifier Head and Dual Outputs

In addition to image generation, the model also predicts a class label. A classification head is attached to the final feature map produced by PixelCNN++. After global average pooling over the spatial dimensions, a fully-connected layer with softmax activation outputs class probabilities. During training, we optimize a joint loss:

$$L_{\text{total}} = L_{\text{gen}} + \lambda L_{\text{class}},$$

where L_{gen} is the generative loss (the NLL from the logistic mixture in Eq. (1)), L_{class} is the cross-entropy loss, and λ (set to 1) balances the two tasks. The network's forward function is modified to support dual outputs: one branch returns the mixture parameters for generation and the other yields class logits for classification.

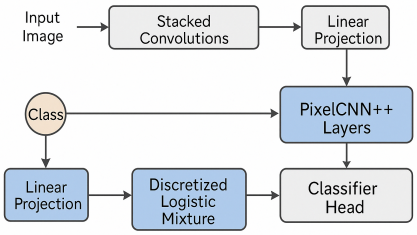


Figure 1: Overview of the Conditional PixelCNN++ architecture

3 Experiments

3.1 Compute Setup and Platform Transitions

The initial development and testing of this project was conducted locally on an Apple M1 MacBook Pro using PyTorch with the Metal Performance Shaders (MPS) backend for GPU acceleration. Although MPS offered noticeable speed improvements over pure CPU execution, it still achieved only around 35% of the throughput observed on an NVIDIA T4 GPU in Google Colab. Furthermore, adapting the codebase to work reliably with MPS required nontrivial changes due to backend-specific issues with autograd, memory usage, and convolution implementations.

Given these limitations, I transitioned the project to Google Colab, which offered more consistent performance with minimal code modifications. During experimentation, I evaluated different GPU tiers available on Colab:

T4: Used for most runs due to its balanced speed and cost efficiency (1.5 credits/hour). It became the baseline for performance comparison. **L4**: Though priced at 2 credits/hour, the L4 accelerated training significantly—roughly 2–3 times faster than T4. For shorter experiments and sweeps, this trade-off proved advantageous. **A100**: Offered marginally faster performance than L4 but consumed 7 credits/hour, making it impractical for extended runs under the available compute quota.

Ultimately, the majority of training and hyperparameter tuning was performed on T4 and L4 instances, balancing compute credits and training efficiency within Colab's constraints.

3.2 Training Setup

I trained the model on a custom dataset consisting of 4,160 labeled training images divided into four categories, along with 520 labeled validation images and 520 unlabeled test images. using the Adam optimizer with a learning rate of 10^{-3}, $\beta_1 = 0.9$, and $\beta_2 = 0.999$. Training was run for 300 epochs—the point at which further training tended to cause overfitting in generation—using a learning rate decay (by a factor of 0.5 every 100 epochs). A batch size of 48 was used (selected for memory considerations). Training metrics were logged using Weights & Biases (W&B).

3.3 Training Dynamics and Performance

Figure 2 shows the training curves. The left panel presents the bits-per-dimension (BPD, i.e. NLL per pixel) decreasing from roughly 5.0 bits at the start to about 2.8 bits at epoch 300, in line with previous reports [1]. The right panel shows the classification accuracy increasing to nearly 80% by epoch 300. (Note that while classification accuracy plateaus after around 250 epochs, the BPD continues to improve slowly.)

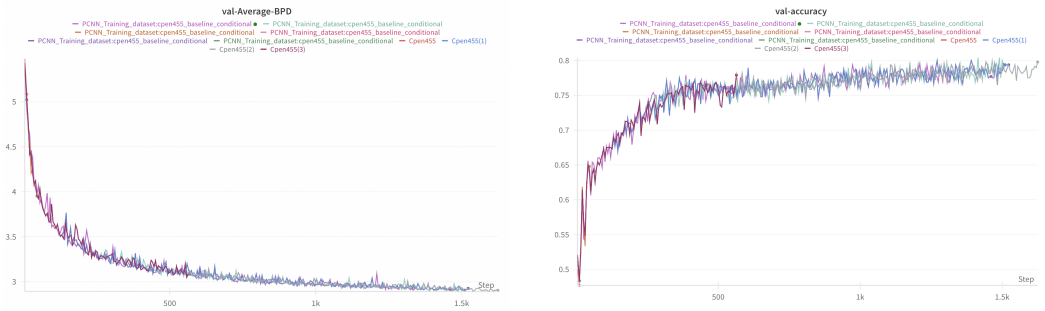
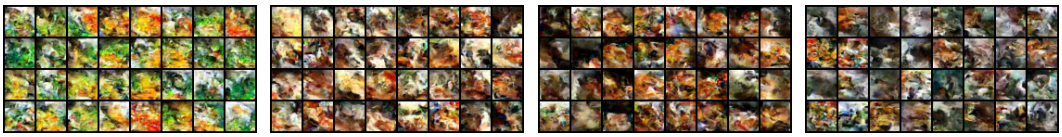


Figure 2: Training curves for Conditional PixelCNN++ on CIFAR-10. **Left:** BPD (normalized NLL) over epochs. **Right:** Classification accuracy over epochs.

After 300 epochs, the final model achieved a test classification accuracy of **79.38%** and a FID of **23.69** [4] (See Appendix Figure 5). Although state-of-the-art discriminative CNNs on CIFAR-10 can exceed 95% accuracy, the results illustrate that a joint generative-discriminative model can perform competitively, despite the inherent trade-off between tasks.



(a) Epoch 100 (b) Epoch 250 (c) Epoch 275 (d) Epoch 300

Figure 3: Class-conditional samples generated at various training epochs. Early samples (epoch 100) appear blurry, while by epoch 300 the samples exhibit recognizable class-specific features—although some reduced diversity (mode collapse) is evident.

3.4 Hyperparameter Ablation

I performed more than 70 training runs, varying key hyperparameters:

- **Model Capacity:** Increasing batch size and max epoch had a positive effect on Accuracy and FID scores (Epoch and Accuracy were especially positively correlated, but overtraining would result in lower FID Scores). I did a wanddb sweep measuring FID and Accuracy scores over epoch, to optimize this. (Appendix Figure 6). Decreasing the number of convolutional filters (e.g. from 80 to 64 or 40) improved both classification accuracy and FID. Deeper networks (more residual blocks per stage) did not yield gains.
- **Mixture Components:** Using $M = 15$ logistic mixture components produced the best generative performance. Reducing M to 5 greatly worsened the Accuracy and FID scores.
- **Conditioning Method:** I compared my input concatenation approach to a learned bias per layer (as in the original PixelCNN++ [1]). The simpler concatenation method performed slightly better in my experiments.
- **Training Stability:** Early bugs (such as inadvertently employing a dummy classifier) resulted in unstable training, but these were corrected in subsequent runs. Additionally, I reduced the frequency of logging to W&B to avoid memory issues. I also submitted my csv file generated on test data to the leaderboard to understand if a similar accuracy was achieved (when compared to validation), it achieved an accuracy of 79.15% & a F1 score of 79.02% which further gave me confidence in the model (See Appendix Figure 7)

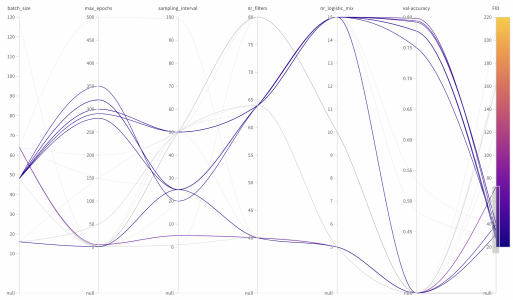


Figure 4: Parallel coordinates plot summarizing various training runs and their outcomes (accuracy, FID, and BPD) under different hyperparameter settings.

4 Conclusion

I have presented a class-conditional PixelCNN++ model capable of both image generation and classification on a Dataset. Achieving nearly 80% classification accuracy and a competitive FID score, this joint model demonstrates that generative architectures can be successfully adapted for discriminative tasks. The extensive experiments show that simple design modifications (e.g., input concatenation for class-conditioning and a straightforward classifier head) can yield effective performance without the added complexity of more involved methods. Future work will explore deeper integration of class information (such as via conditional batch normalization) and techniques to further promote sample diversity.

References

[1] T. Salimans, A. Karpathy, X. Chen, and D. P. Kingma. *PixelCNN++: Improving the PixelCNN with Discretized Logistic Mixture Likelihood and Other Modifications*. ICLR 2017.

[2] A. van den Oord, N. Kalchbrenner, and K. Kavukcuoglu. *Pixel Recurrent Neural Networks*. ICML 2016.

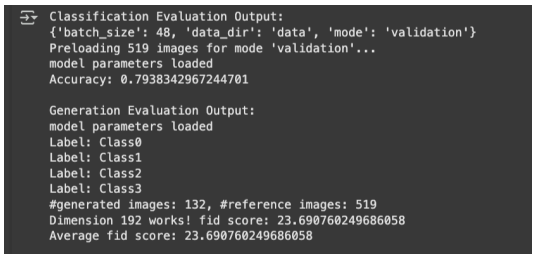
[3] A. van den Oord, N. Kalchbrenner, O. Vinyals, et al. *Conditional Image Generation with PixelCNN Decoders*. NIPS 2016.

[4] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter. *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*. NIPS 2017.

[5] L. Biewald. *Experiment Tracking with Weights and Biases*. 2020. Available: `https://wandb.ai/`.

[6] GitHub. *CPEN455HW-2024W2 Course Project: Conditional PixelCNN++ for Image Classification and Generation*. Available: `https://github.com/DSL-Lab/CPEN455HW-2024W2`.

Appendix



```
Classification Evaluation Output:
{'batch_size': 48, 'data_dir': 'data', 'mode': 'validation'}
Preloading 519 images for mode 'validation'...
model parameters loaded
Accuracy: 0.7938342967244701

Generation Evaluation Output:
model parameters loaded
Label: Class0
Label: Class1
Label: Class2
Label: Class3
#generated images: 132, #reference images: 519
Dimension 192 works! fid score: 23.690760249686058
Average fid score: 23.690760249686058
```

Figure 5: Screenshot of best Accuracy and FID score achieved by the final model.

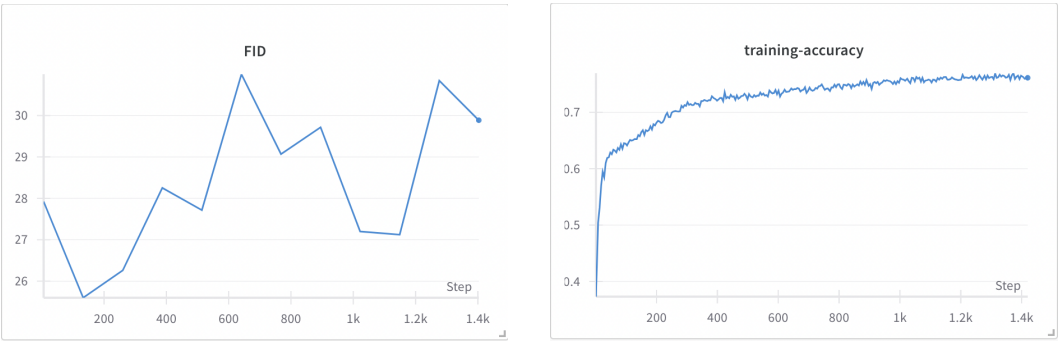


Figure 6: Wanddb Sweep for params over Epoch. **Left:** FID score. **Right:** Training Accuracy.

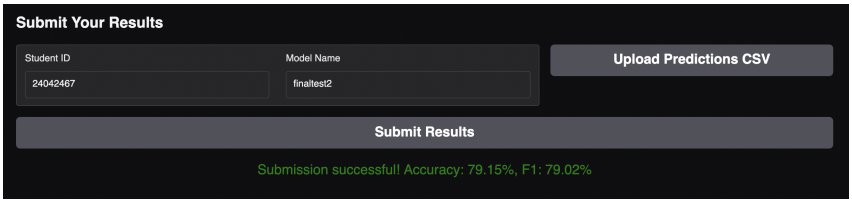


Figure 7: Leaderboard submission (To get information on accuracy when tested on ”test” data.

LLM Assistance Acknowledgment

Throughout this project, I have used LLM's (Chat GPT 4o and 03) to support various stages of development. Here's a brief overview:

- Repository Parsing: Clarified the GitHub repository, README, and guidance documents, providing structure insights and best practices.
- Deep Research: Offered insights into academic publications, libraries, and modern methodologies that shaped the project's approach.
- Debugging: Identified error sources, explained error messages, and suggested improvements for code efficiency.
- Experiment Tracking & Data Scripting: Assisted with setting up Weights & Biases (WandB) and developing CSV scripts for test data, ensuring proper logging and data processing.
- Google Colab Training: Recommended hyperparameter tuning, memory optimization, and GPU utilization enhancements for faster, more stable training.
- LaTeX Report Drafting: Helped organize the report's structure and improve clarity, formatting, and referencing.

Note: Detailed screenshots of the LLM usage are provided in a separate PDF titled "LLM Usage."